Real-Time Shape Detection and Frame-by-Frame Segmentation Pipeline

Model Performance Whitepaper
Artificial Intelligence Internship, Progree
Prepared by: Ubaid Ullah

Abstract

This whitepaper presents a classical computer vision pipeline built with OpenCV that detects, classifies and counts geometric shapes (circles, rectangles, triangles) in video, frame by frame. The pipeline applies a Gaussian filter and an adaptive Gaussian threshold to segment objects under uneven lighting, extracts contours, classifies each contour by polygon approximation and circularity, and overlays dynamic per-class counts on every frame. It was evaluated on a 240-frame synthetic video with known ground truth, which allows exact measurement of detection quality. The system reached an F1-score of **0.901** (precision 0.936, recall 0.870) at a mean latency of **4.1 ms per frame** (about 245 FPS) on CPU. Errors were traced almost entirely to touching or overlapping objects that merge into one blob. A noise-versus-blur ablation shows the Gaussian filter is essential at high noise.

1. Introduction

Object detection and segmentation are core computer vision tasks. Deep models such as YOLO are powerful but need trained weights and more compute. For well-contrasted objects, a classical pipeline built from filtering, thresholding and contour analysis can be fast, transparent and easy to debug. The objective of this project is to build such a pipeline that works in real time, tracks target shapes through a video stream, overlays live counts, and is measured rigorously frame by frame.

The main contributions are: (1) an end-to-end OpenCV pipeline that runs on a video file or a webcam; (2) a synthetic benchmark with exact ground truth; (3) a frame-by-frame evaluation of accuracy and latency; and (4) an ablation study of noise level against Gaussian kernel size.

2. Methodology

Each frame passes through the following stages. The same code runs on a recorded video or a live webcam stream (`--source 0`).

2.1 Pipeline stages

Stage	OpenCV function	Purpose
1. Grayscale	cvtColor	Remove colour, reduce data 3x
2. Gaussian filter	GaussianBlur (5x5)	Suppress sensor noise before thresholding
3. Adaptive threshold	adaptiveThreshold (block 31, C 8)	Local threshold, robust to uneven lighting
4. Morphology	morphologyEx open + close (3x3)	Remove specks, fill small holes
5. Contours	findContours (external)	Extract shape outlines (pixel bounds)
6. Filtering	contourArea (350 to 20000 px)	Reject noise blobs and huge regions
7. Classification	approxPolyDP + circularity	Label circle, rectangle, triangle
8. Overlay	rectangle, putText	Draw boxes and dynamic count vector

2.2 Why adaptive thresholding

A global threshold uses one brightness cut-off for the whole image, which fails when lighting varies. Adaptive thresholding computes a separate threshold for every pixel from the Gaussian-weighted mean of its neighbourhood (block size 31 pixels) minus a constant C. The test video has a deliberate brightness gradient across the background, so this choice is necessary.

2.3 Shape classification rule

For each contour the perimeter and area give the circularity `4*pi*area / perimeter^2`, which equals 1.0 for a perfect circle. The contour is simplified with approxPolyDP (epsilon = 4% of the perimeter) and the vertex count v is read: circularity above 0.82 with more than 5 vertices gives **circle**; v = 3 gives **triangle**; v = 4 gives **rectangle**; anything else is **other** (not counted in the class totals). The counts are drawn as a text block in the top-left corner and update every frame.

2.4 Core code

```
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
g = cv2.GaussianBlur(gray, (5, 5), 0)
mask = cv2.adaptiveThreshold(g, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY_INV, 31, 8)
cnts, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
```

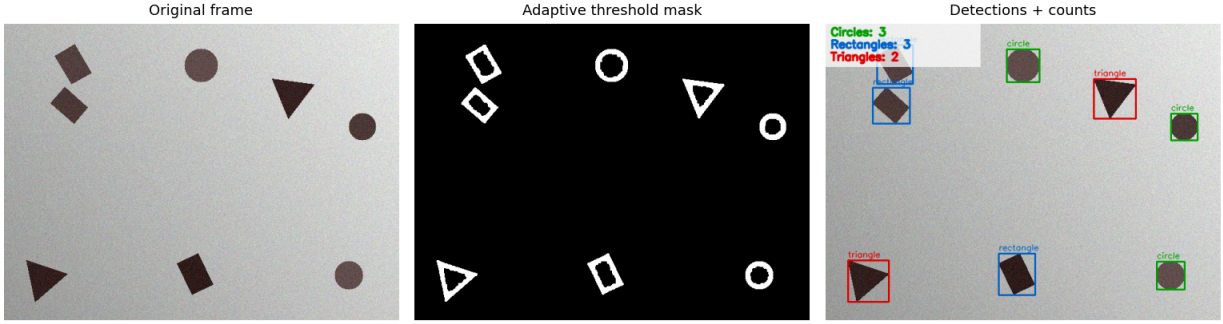


Figure 1. Original frame, adaptive-threshold mask, and annotated output with live counts.

3. Experimental Setup

Dataset. A synthetic 640x480, 30 FPS, 240-frame video was generated with fixed random seed 7. It contains 8 moving, rotating, bouncing objects (3 circles, 3 rectangles, 2 triangles), a brightness gradient background and Gaussian pixel noise (sigma = 12). Because the generator records every object's position, the ground truth is exact for all 1,920 object instances.

Metrics. A detection is a true positive (TP) if it has the same class as a ground-truth object and its centroid lies within 25 pixels (greedy one-to-one matching). Unmatched detections are false positives (FP); unmatched ground-truth objects are false negatives (FN). Precision = TP/(TP+FP), recall = TP/(TP+FN), F1 is their harmonic mean. *Exact-count accuracy* is the share of frames where all three class counts equal the truth. Latency is measured per frame with `time.perf_counter` on CPU (single thread, no GPU).

4. Results

4.1 Overall detection quality

Metric	Value
Frames evaluated	240
True positives / FP / FN	1670 / 115 / 250
Precision	0.936
Recall	0.870
F1-score	0.901
Exact-count accuracy (all 3 classes)	45.4%

4.2 Per-class results

Class	Precision	Recall	F1	TP	FN
Circle	1.000	0.926	0.962	667	53
Rectangle	1.000	0.811	0.896	584	136
Triangle	1.000	0.873	0.932	419	61

Per-class precision is 1.000: whenever the system labels an object as a circle, rectangle or triangle at the right place, the label is correct. All 115 overall false positives are blobs labelled *other*, which come from merged objects. Circles are easiest (recall 0.926), because their silhouette stays the same under rotation. Rectangles have the lowest recall (0.811).

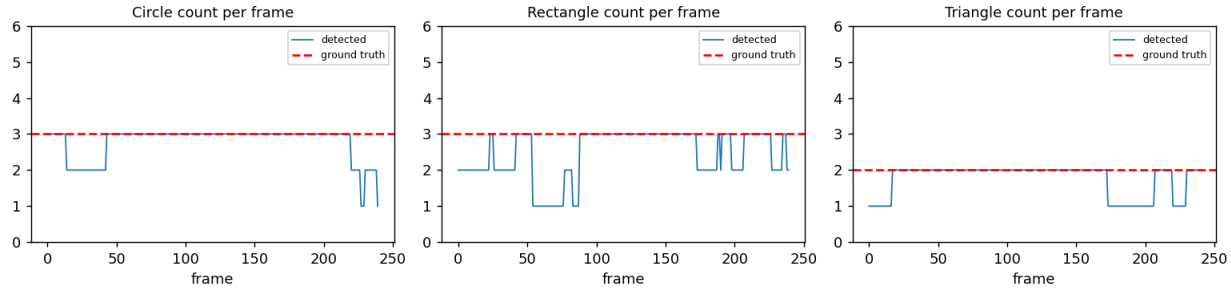


Figure 2. Detected count per class versus ground truth for every frame.

4.3 Frame-by-frame latency

The mean total time is **4.09 ms** per frame and the 95th percentile is 5.31 ms, so even the slow frames are far inside the 33 ms budget of a 30 FPS stream. This gives roughly 245 FPS of pure processing speed, about 8 times faster than real time, which leaves room for tracking, video decoding and display. Figure 3 shows where time is spent. The adaptive threshold is the most expensive processing stage (about 1.9 ms of the 4.1 ms), followed by contour extraction (0.4 ms) and morphology (0.3 ms); grayscale conversion and blur cost under 0.2 ms each. The remaining time (about 1.1 ms) is spent on drawing boxes, labels and the count overlay. Latency is stable across frames; spikes come from operating-system scheduling.

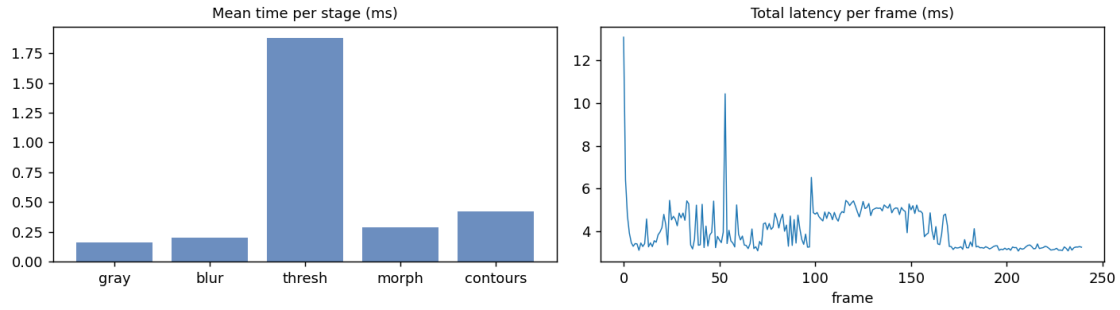


Figure 3. Mean time per pipeline stage (left) and total latency per frame (right).

4.4 Ablation: noise level and Gaussian kernel size

The video generator was re-run at three noise levels (sigma 5, 12, 25) with 100 frames each, and the pipeline was evaluated with Gaussian kernel sizes 1 (no filter), 3, 5 and 9.

Noise sigma	k=1 (no blur)	k=3	k=5	k=9
5	0.851	0.856	0.859	0.858
12	0.853	0.855	0.858	0.858
25	0.777	0.859	0.860	0.860

Table 3. F1-score for each noise level and Gaussian kernel size.

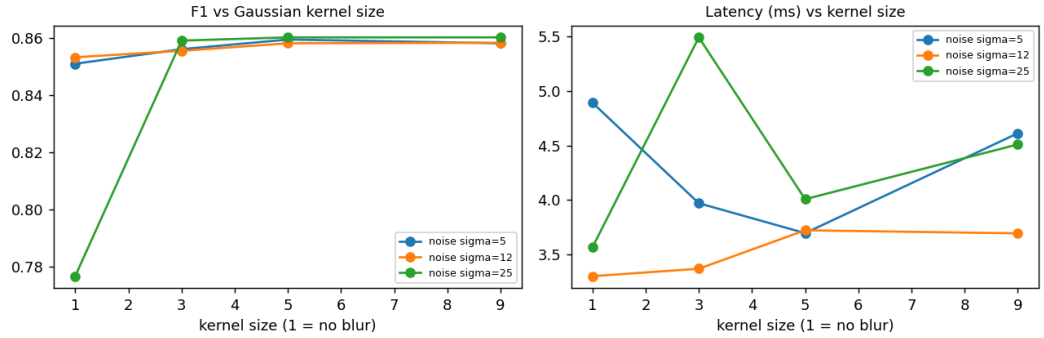


Figure 4. F1-score and latency against kernel size for three noise levels.

At low and medium noise the filter makes almost no difference (F1 about 0.85 to 0.86). At sigma = 25, removing the filter drops F1 from 0.860 to **0.777**, because noisy pixels create many small false contours; any kernel of 3 or larger restores performance. A 5x5 kernel is the best choice: it gives the full benefit with no measurable accuracy loss and negligible extra time. Latency differences between kernels are within timing noise.

5. Discussion

5.1 Error analysis

Exact-count accuracy is only 45.4% even though F1 is 0.90. The reason is that the metric is strict: a single merged pair in a frame makes that whole frame wrong. In 180 of the 240 frames at least two objects are within 70 pixels of each other, and in 110 frames the pipeline produced at least one blob labelled *other*. When two shapes touch, the external contour of the combined blob has an irregular polygon, so it matches no class, and both objects are lost (false negatives) while one false positive appears. This explains why recall (0.870) is lower than precision (0.936) and why all false positives are of type *other*.

5.2 Classical pipeline versus deep learning

Aspect	This pipeline	YOLO-type detector
Speed (CPU)	4.1 ms/frame	tens of ms, usually needs GPU
Training data	None	Thousands of labelled images
Handles overlap	Poorly	Well (learned occlusion)
Handles texture / real scenes	Poorly	Well
Explainability	Every step is visible	Black box

The classical approach is the right tool for simple, high-contrast objects (for example parts on a conveyor belt, markers, or shape-based games) where speed and transparency matter. For general real-world objects, a pre-trained YOLO model would be the better choice, and it can be dropped into the same frame loop to replace stages 2 to 7.

5.3 Limitations

(1) Results are from a synthetic video with dark shapes on a light background; real footage has shadows, texture and colour variation and would lower accuracy. (2) Overlapping shapes are not separated. (3) Classification relies on fixed thresholds (circularity 0.82, 4% polygon epsilon) tuned by hand. (4) There is no object tracking, so counts are per frame, not unique objects over time. (5) Latency was measured on a single CPU and will differ on other hardware.

6. Future Work

Planned improvements: a distance-transform plus watershed step to split touching objects; centroid tracking to give stable object IDs and cumulative counts; evaluation on real recorded video; replacing the classifier with a pre-trained YOLO model and comparing mAP and latency on the same video; and automatic threshold tuning.

7. Conclusion

A lightweight OpenCV pipeline built from a Gaussian filter, adaptive thresholding and contour analysis detected and classified shapes in real time with an F1-score of 0.90 at 4.1 ms per frame. Per-class precision was perfect, the Gaussian filter proved essential under heavy noise, and nearly all remaining errors came from touching objects. The project shows that a simple, explainable pipeline is fast and accurate for clean scenes, and it identifies clear next steps for harder ones.

8. How to Reproduce

```
pip install -r requirements.txt
python make_video.py   # creates data/test_video.mp4 + ground truth
python cv_pipeline.py   # annotated video + per-frame metrics
python experiments.py   # evaluation, ablation, figures
python cv_pipeline.py --source 0   # live webcam
```

References

[1] OpenCV documentation: Image Thresholding, Contour Features, Structural Analysis (docs.opencv.org).
[2] Bradski, G. The OpenCV Library. Dr. Dobb's Journal, 2000.
[3] Redmon, J. et al. You Only Look Once: Unified, Real-Time Object Detection. CVPR, 2016.